```
import torch
import numpy as np
import onnx
from ultralytics import YOLO
import tensorrt as trt
import pycuda.autoinit
import pycuda.driver as cuda
import time
from scipy.spatial.distance import cosine


onnx_output_path = "model_fp32.onnx"
engine_file_32 = "model_FP32.engine"
engine_file_16 = "model_FP16.engine"
```

Экспортируем предобученную модель YOLOv8 в формат ONNX для последующей конвертации в TensorRT:

```
# Экспорт модели YOLOv8 в ONNX
def export_yolo_to_onnx():
    model = YOLO('yolov8n.pt')
    model.export(format='onnx', dynamic=False, simplify=True, opset=12,
imgsz=640)
    print(f"ONNX модель сохранена как {onnx_output_path}")
```

Создаём и сохраняем движок TensorRT из ONNX-модели, с поддержкой режимов FP32 или FP16:

```
def build_engine(onnx_path, engine_path, use_fp16=False):
    logger = trt.Logger(trt.Logger.WARNING)
    builder = trt.Builder(logger)
    network_flags = 1 <<
int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
    network = builder.create_network(network_flags)
    parser = trt.OnnxParser(network, logger)

    with open(onnx_path, 'rb') as f:
      parser.parse(f.read())

    config = builder.create_builder_config()
```

```
    if use_fp16 and builder.platform_has_fast_fp16:
      config.set_flag(trt.BuilderFlag.FP16)

    profile = builder.create_optimization_profile()
    input_tensor = network.get_input(0)
    shape = input_tensor.shape
    profile.set_shape(input_name, min=(1, 3, 640, 640), opt=(4, 3, 640,
640), max=(8, 3, 640, 640))
    config.add_optimization_profile(profile)

    serialized_engine = builder.build_serialized_network(network,
config)
    if serialized_engine is None:
        raise RuntimeError("Failed to build serialized engine")

    with open(engine_path, 'wb') as f:
        f.write(serialized_engine)

    print(f"Движок {'FP16' if use_fp16 else 'FP32'} создан:
{engine_path}")
```

Выполняем инференс модели в TensorRT на GPU, используя сериализованный движок и входные данные.

```
#  Инференс с TensorRT
def infer_tensorrt(engine_path, input_data):
    logger = trt.Logger(trt.Logger.WARNING)
    with open(engine_path, "rb") as f:
        engine_data = f.read()
    runtime = trt.Runtime(logger)
    engine = runtime.deserialize_cuda_engine(engine_data)
    context = engine.create_execution_context()

    input_name = None
    output_names = []
    for i in range(engine.num_io_tensors):
        name = engine.get_tensor_name(i)
        mode = engine.get_tensor_mode(name)
        if mode == trt.TensorIOMode.INPUT:
            input_name = name
        elif mode == trt.TensorIOMode.OUTPUT:
            output_names.append(name)
```

```
    context.set_input_shape(input_name, input_data.shape)

    bindings = []
    d_input = cuda.mem_alloc(input_data.nbytes)
    bindings.append(int(d_input))
    context.set_tensor_address(input_name, int(d_input))

    output_buffers = {}
    d_outputs = []
    for name in output_names:
        shape = context.get_tensor_shape(name)
        dtype = trt.nptype(engine.get_tensor_dtype(name))
        host_buffer = np.empty(shape, dtype=dtype)
        output_buffers[name] = host_buffer
        device_buffer = cuda.mem_alloc(host_buffer.nbytes)
        d_outputs.append(device_buffer)
        bindings.append(int(device_buffer))
        context.set_tensor_address(name, int(device_buffer))

    stream = cuda.Stream()
    cuda.memcpy_htod_async(d_input, input_data, stream)
    context.execute_async_v3(stream_handle=stream.handle)

    for name, device_buffer in zip(output_names, d_outputs):
        host_buffer = output_buffers[name]
        cuda.memcpy_dtoh_async(host_buffer, device_buffer, stream)

    stream.synchronize()
    d_input.free()
    for buf in d_outputs:
        buf.free()

    return output_buffers

# Объединение выходов TensorRT
def combine_trt_outputs(outputs):
    return list(outputs.values())[0]
```

Метрики точности для вывода ответа: max, медиана, коэф. Отиаи, среднеквадратичная ошибка

```
def calculate_metrics(ref, test):
    abs_diff = np.abs(ref - test)
```

```
    return {
        "max_diff": np.max(abs_diff),
        "mean_diff": np.mean(abs_diff),
        "cosine_sim": 1 - cosine(ref.flatten(), test.flatten()),
        "mse": np.mean((ref - test) ** 2)
    }


# Экспорт модели в ONNX
export_yolo_to_onnx()

# Создание движков
build_engine(onnx_output_path, engine_file_32, use_fp16=False)
build_engine(onnx_output_path, engine_file_16, use_fp16=True)

# Какой сейчас юзаем процессор
print(torch.cuda.is_available())
print("GPU:", pycuda.autoinit.device.name())

# Подготовка входных данных
pt_model = YOLO('yolov8n.pt').model.eval()
dummy_input = torch.randn(1, 3, 640, 640)
input_np = dummy_input.cpu().numpy()

# Теплый запуск
_ = infer_tensorrt(engine_file_32, input_np)

# Измерение времени FP32
trt_fp32_times = []
for _ in range(100):
    start = time.time()
    _ = infer_tensorrt(engine_file_32, input_np)
    trt_fp32_times.append(time.time() - start)
avg_trt_fp32 = np.mean(trt_fp32_times) * 1000

# Измерение времени FP16
_ = infer_tensorrt(engine_file_16, input_np)
trt_fp16_times = []
for _ in range(100):
    start = time.time()
    _ = infer_tensorrt(engine_file_16, input_np)
    trt_fp16_times.append(time.time() - start)
avg_trt_fp16 = np.mean(trt_fp16_times) * 1000
```

```
# Выход PyTorch
with torch.no_grad():
    pt_outputs = pt_model(dummy_input)
    pt_output = pt_outputs[0].cpu().numpy() if isinstance(pt_outputs,
tuple) else pt_outputs.cpu().numpy()

# Сравнение выходов
trt_fp32_combined = combine_trt_outputs(infer_tensorrt(engine_file_32,
input_np))
trt_fp16_combined = combine_trt_outputs(infer_tensorrt(engine_file_16,
input_np))

metrics_fp32 = calculate_metrics(pt_output, trt_fp32_combined)
metrics_fp16 = calculate_metrics(pt_output, trt_fp16_combined)

# Вывод результатов
print("\n" + "="*50)
print("Результаты производительности:")
print(f"TensorRT FP32: {avg_trt_fp32:.2f} мс")
print(f"TensorRT FP16: {avg_trt_fp16:.2f} мс")
print(f"Ускорение FP16: {avg_trt_fp32 / avg_trt_fp16:.2f}x")

print("\n" + "="*50)
print("Сравнение точности:")
print("FP32:")
for k, v in metrics_fp32.items():
    print(f"{k:>12}: {v:.6f}")
print("\nFP16:")
for k, v in metrics_fp16.items():
    print(f"{k:>12}: {v:.6f}")

print("\nКонвертация и тестирование завершены!")
```

Вывод:
_____Тестирование производительности и точности_____
ONNX модель сохранена как model_fp32.onnx
True
GPU: Tesla T4

==========

Результаты производительности:
TensorRT FP32: 43.26 мс
TensorRT FP16: 39.47 мс
Ускорение FP16: 1.10x

==========
Сравнение точности:
FP32:
max_diff: 0.000671
mean_diff: 0.000002
cosine_sim: 1.000000
mse: 0.000000

FP16:
max_diff: 3.071899
mean_diff: 0.004656
cosine_sim: 1.000000
mse: 0.001343

Конвертация и тестирование завершены!

ИТОГО:

	PyTorch CPU	.onnx (FP32) CPU	.onnx (FP16) CPU	.engine (FP32) GPU	.engine (FP16) GPU
Inference (мс)	86.53	64.32	76.80	43.26	39.47
Max_diff (vs PyTorch)	_	0.0005187988	1.25	0.000671	3.071899

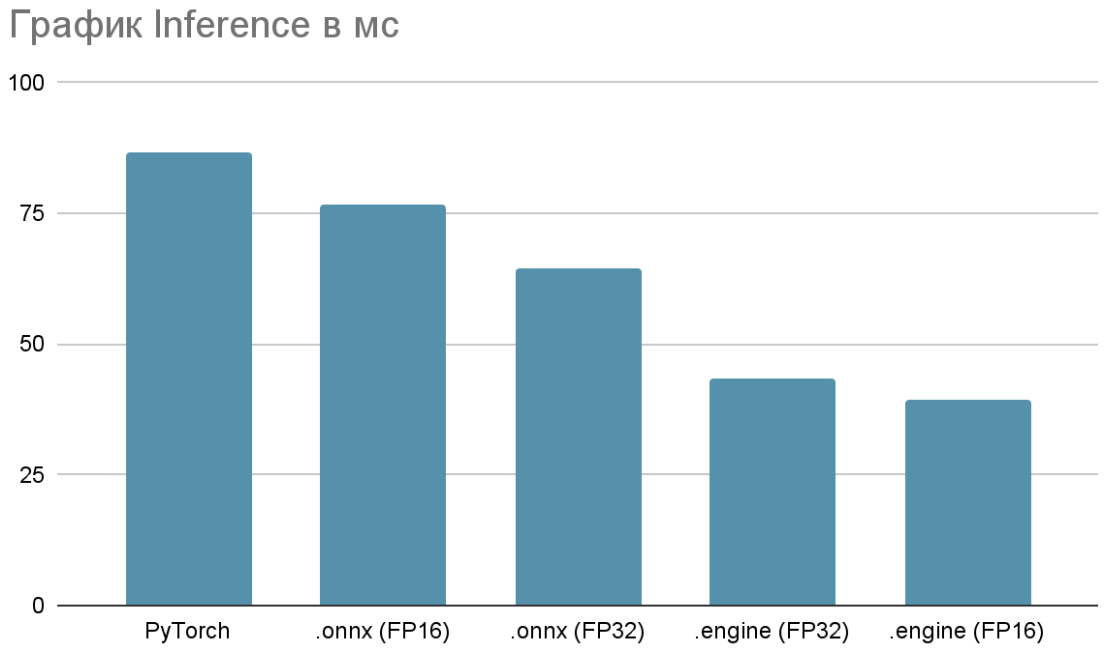
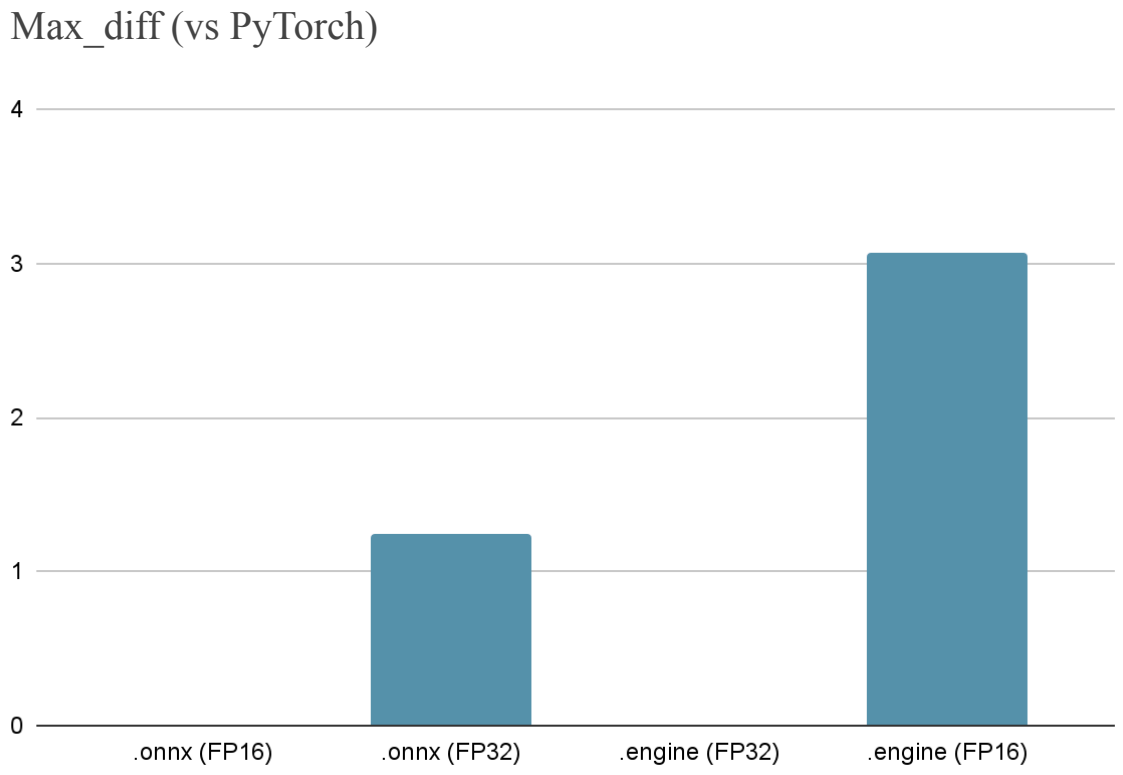


График Inference в мс
100
75
50
25
0
PyTorch
.onnx (FP16)
.onnx (FP32)
.engine (FP32)
.engine (FP16)


Max_diff (vs PyTorch)
4
3
2
1
0
.onnx (FP16)
.onnx (FP32)
.engine (FP32)
.engine (FP16)